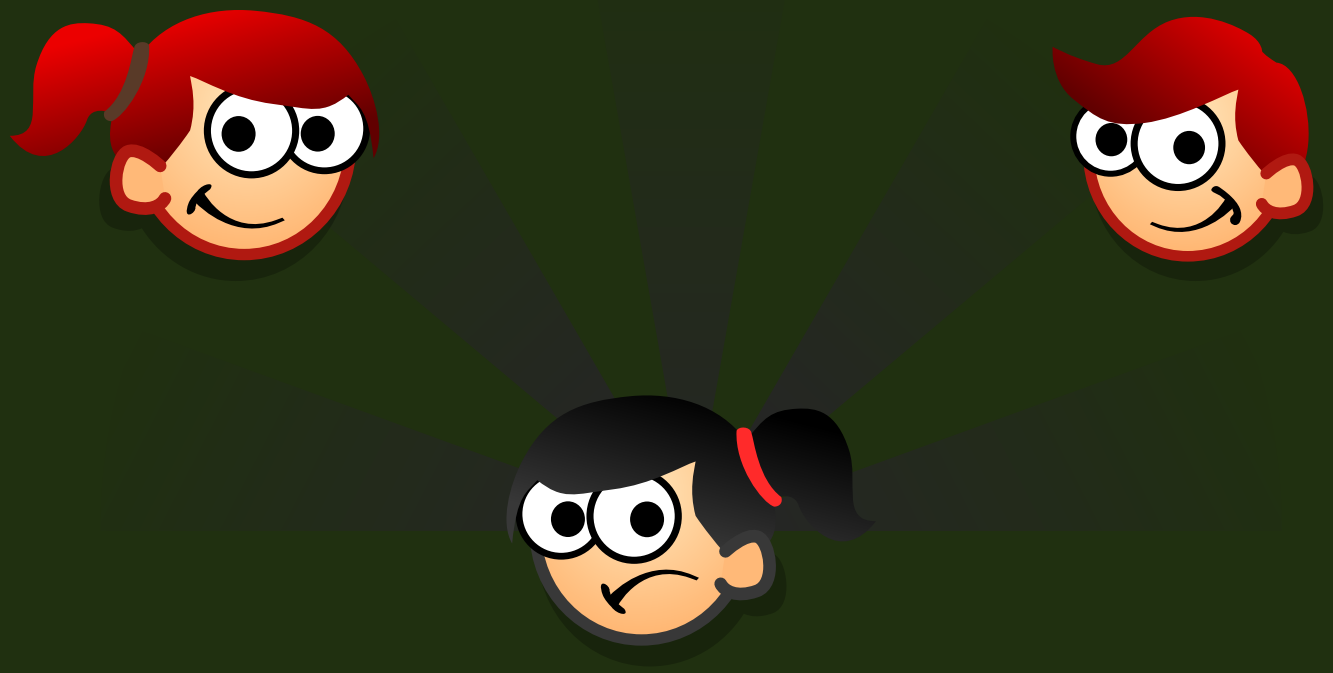


Public-Key Infrastructure

Park Jae Heung , Aaron Peng, Xia Tianyi, Li Ang

Table of contents

1 Certificates Authority

2 Encryption in Public Key Infrastructure

3 PKI Application in Linux

1. Certificates Authority

PKI and CA Certificates Authority

Let's imagine a scenario where Alice and Bob are part of a vast network. In this network, each participant has specific public and private keys for designated cryptosystems or signature schemes. In such an environment, it's crucial to have a method to verify the public keys of other network members. This necessitates a public-key infrastructure (PKI).

Typically, we assume the existence of a trusted certification authority (CA), which signs the public keys of all network participants. The CA's public verification key, known as verCA, is assumed to be universally known within the network. While this setting might be oversimplified, it helps us focus on the design of the schemes.

A network member's certificate will include identifiable information about the individual (like their name, email address, etc.), their public key(s), and the CA's signature on that data.

Steps to Issue a Certificate to Alice [3]

1. The Certification Authority (CA) verifies Alice's identity using traditional identification methods such as a birth certificate, passport, etc. Following this, the CA creates a string, referred to as ID(Alice), which includes Alice's identification details.
2. A private signing key for Alice, denoted as sigAlice, and a corresponding public verification key, verAlice, are established.
3. The CA generates its signature, represented as:

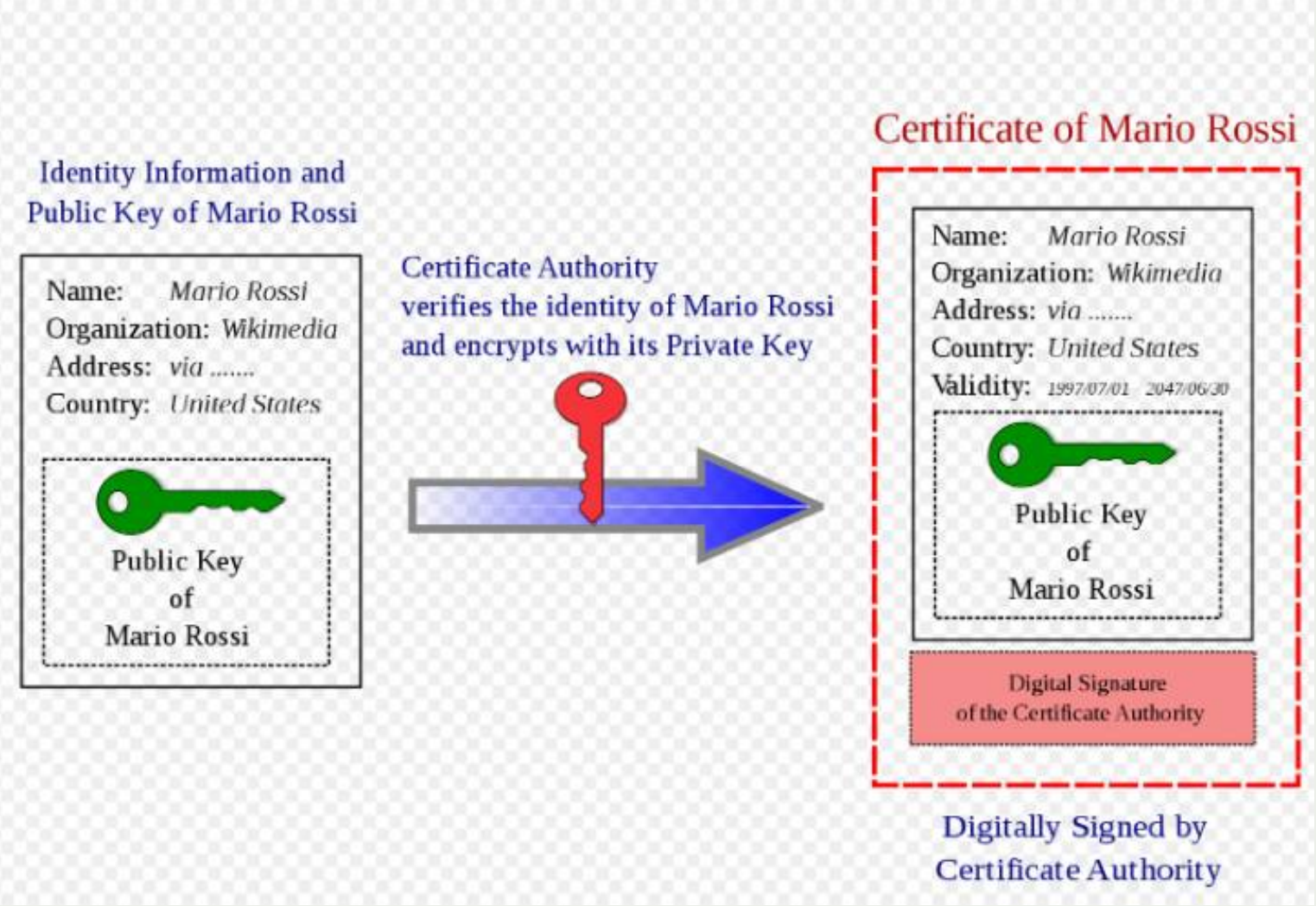
$$s = \mathrm{sig}_{\mathrm{CA}}(ID_{\mathrm{Alice}} \parallel ver_{\mathrm{Alice}}) \tag{1}$$

This signature is based on Alice's identity string and verification key. The certificate, represented as:

$$\mathrm{Cert}(\mathrm{Alice}) = (ID_{\mathrm{Alice}} \parallel ver_{\mathrm{Alice}} \parallel s) \tag{2}$$

is then provided to Alice, along with Alice's private key, sig_{Alice}.

PublicKeyCertificateDiagram



By I, Giaros, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=2260308

2. Encryption in Public Key Infrastructure

Two ways of Public Key Encryption

Two ways of Public Key Encryption

- Sign Then Encrypt
- Encrypt Then Sign

Sign Then Encrypt

Imagine Alice needs to send a signed and encrypted message x to Bob.

1. Alice generates her signature $y = \mathrm{sig}_{Alice}(x, ID(Bob))$.
2. Alice then creates the ciphertext $z = e_{Bob}(x, y, ID(Alice))$, and sends it to Bob.

Encrypt Then Sign

Consider that ID_{Alice} and ID_{Bob} are public ID strings of a fixed length for Alice and Bob, respectively.
Imagine Alice needs to send a signed and encrypted message x to Bob.

1. Alice generates the ciphertext z using the formula $e_{Bob}(x, ID_{Alice})$.
2. Alice then creates her signature y using the formula $sig_{Alice}(z, ID_{Bob})$, and sends the tuple (z, y, ID_{Alice}) to Bob.

3. PKI Application in Linux

HTTPS

Currently, most of the world uses web server with HTTPS protocol. Its predecessor, HTTP, is unsafe because it sends data in plain text. HTTPS, on the other hand, can encrypt the traffic between web server and client.
The easiest way to do this to let the server have public key and private key. It can be done in the following way:

- The client send a random AES key encrypted with server's public key.
- The server decrypts the key with its private key.
- As now both sides now have the key, a secure communication channel is established with symmetric encryption.

HTTPS

The problem with this approach is that it cannot prevent man-in-the-middle attack. Anyone between the client and the server can pretend to be the server.
The attacker can replace the server's public key with its own key, so the encrypted AES key can be decrypted with attacker's private key.
The attacker can then use the leaked key to decrypt all the traffic between the server and the client.
The CA certificate was invented to prevent this kind of attack.

HTTPS

The CA server contains a public key (decryption) and a private key (encryption). The following things are being done:

- The web server sends its ID and public key (certificate signing request) to the CA server.
- The CA server verify these information and sends back a certificate that contains its ID, unencrypted web server public key, and web server public key encrypted (signed) with CA private key.
- When establishing an connection, the web server sends the certificate instead of the public key to the client. The client extracts the encrypted and unencrypted key, then it decrypts the encrypted key with a CA public key (chosen by checking the CA's ID contained in the certificate) and check whether it matchs the unencrypted key.
- If the key matches, the client can use the web server public key to encrypt the AES key, send it to the server, and establish a secure communication channel.

Linux HTTPS Web Server

The following in an example of self-signed HTTPS web server on Linux:

Server: Linux container running Ubuntu 22.04, IP: 10.10.10.159
Client: Arch Linux with Google Chrome browser

```
# Create a Certificate Authority
# Generate CA private key
openssl genrsa -des3 -out myCA.key 2048
# Generate root certificate (CA public key and other info)
openssl req -x509 -new -nodes -key myCA.key -sha256 -days 365 -out myCA.pem

# Create CA-signed certs
# Generate a private key for web server
openssl genrsa -out testcrt.com.key 2048
# Create a certificate-signing request (web server public key and other info)
openssl req -new -key testcrt.com.key -out testcrt.com.csr
# Create a config file for the extensions (additional info)
nano testcrt.com.ext
```

Content of testcrt.com.ext:

```
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
keyUsage = digitalSignature, nonRepudiation, keyEncipherment, dataEncipherment
subjectAltName = @alt_names
[alt_names]
DNS.1 = testcrt.com # Server domain name
IP.1 = 10.10.10.159 # Server IP, optional
```

After that, create the signed certificate.

```
# Create the signed certificate with CA private key, CA info, and CSR
openssl x509 -req -in testcrt.com.csr -CA myCA.pem -CAkey myCA.key \
-CAcreateserial -out testcrt.com.crt -days 365 -sha256 -extfile testcrt.com.ext
# Put the certificate and private key to the desired location
sudo cp testcrt.com.crt /etc/certificate/
sudo cp testcrt.com.key /etc/certificate/
```

Linux HTTPS Web Server

Now we have the certificate and key required for the server. Suppose apache web server is already installed and configured, we can configure it to use the certificate and the key.
Content of /etc/apache2/sites-available/default-ssl.conf:

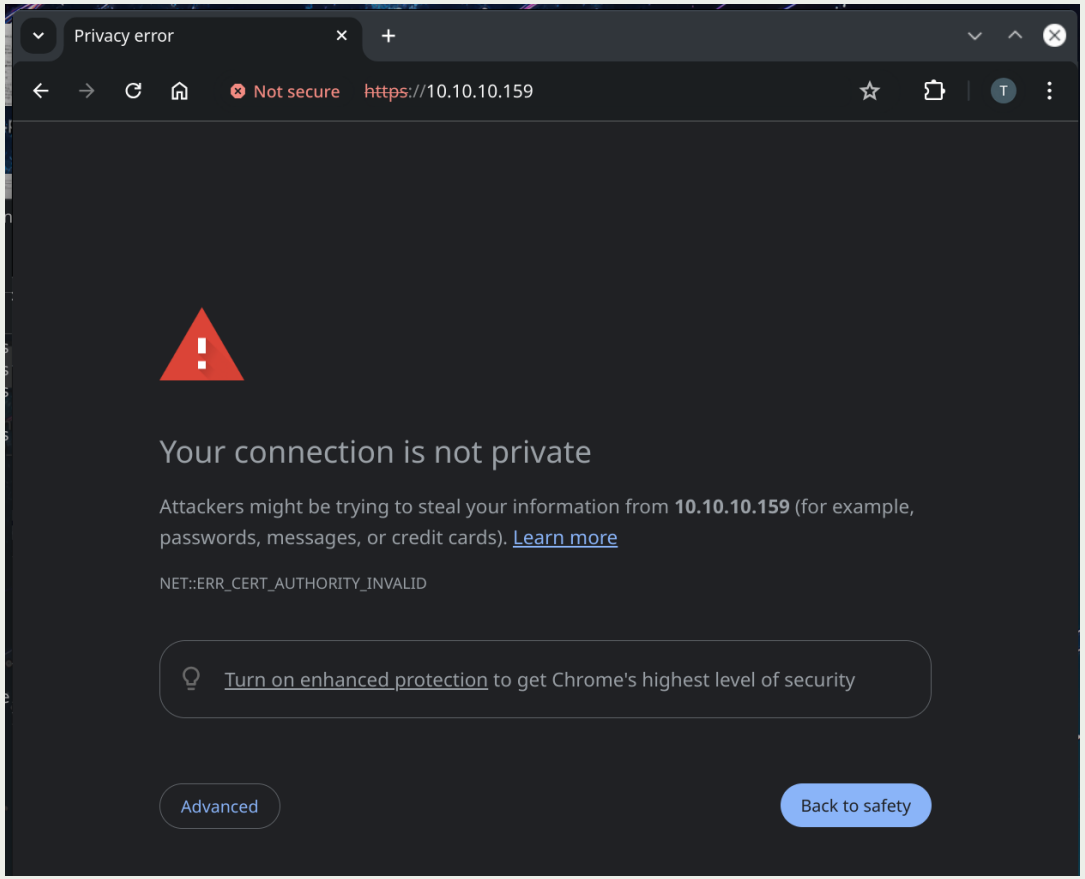
```
default-ssl.conf: # Change/Add following entry

<IfModule mod_ssl.c>
    <VirtualHost _default_:443>
        ServerAdmin ap1028@sjtu.edu.cn
        ServerName testcrt.com
        SSLCertificateFile         /etc/certificate/testcrt.com.crt
        SSLCertificateKeyFile      /etc/certificate/testcrt.com.key
```

Now reconfigure and restart the web server.

Linux HTTPS Web Server

Now the server is configured, we can get to the client and access https://10.10.10.159 (Or https://testcrt.com if client /etc/hosts include the line "10.10.10.159 testcrt.com")
We will be greeted with a warning from the browser:



Linux HTTPS Web Server

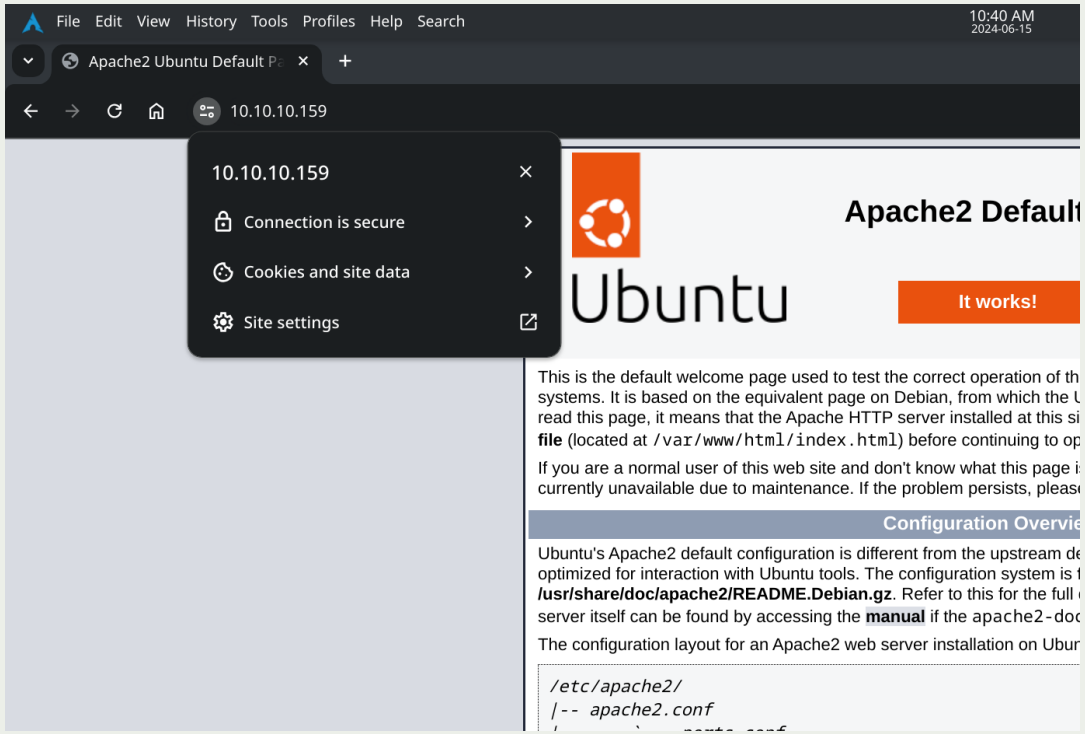
This is because our client does not know our CA, as it is a custom self-signed CA instead of the common ones trusted and used over the internet. The browser cannot verify whether our server can be trusted. It is possible to ignore the warning and proceed to the website because the client can still extract the public key of the web server in plaint text with the certificate it received without the public key. The traffic will still be encrypted. However, it can be attacked by man-in-the-middle attack as the client is not verifying the server public key at all.

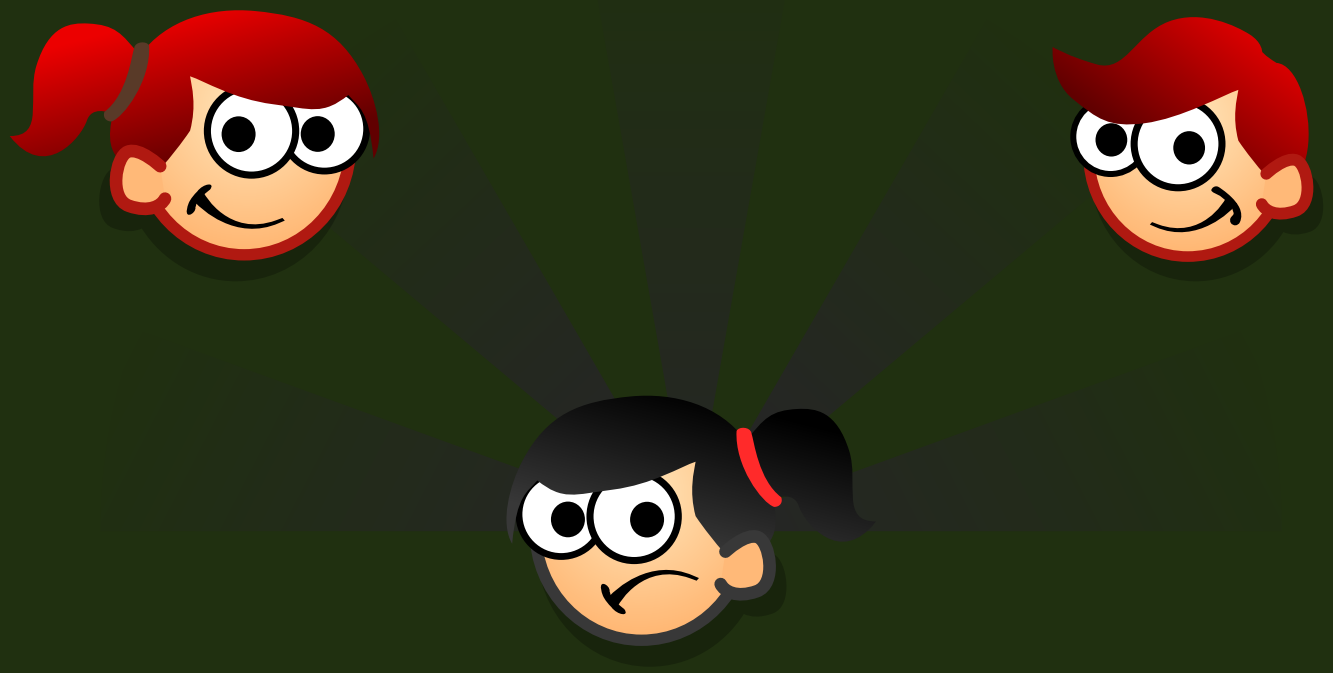
We need the following additional configuration to the browser to make it know the CA, trust the CA, and have its public key.

- Download myCA.pem from the server
- Go to chrome://settings/certificates
- Go to Authorities tab, import and trust myCA.pem.

Linux HTTPS Web Server

Now everything should just work.
The connection is secure, and no more risk of man-in-the-middle attack.





Thank you!

Biblography

- Manuel Charlemagne (2024) ECE475 Introduction to Cryptography
- Jonathan Katz and Yehuda Lindell (2007) Introduction to Modern Cryptography
- Douglas R. Stinson and Maura B. Paterson (2019) Cryptography Theory and Practice Fourth Edition
- https://stackoverflow.com/questions/7580508/getting-chrome-to-accept-self-signed-localhost-certificate
- Apache2 manual: https://httpd.apache.org/docs/2.4/